

```
"""
```

```
Business Analytics Tool - Version 1.1 (Stability)
```

```
Upload a CSV or Excel file -> automatic analysis -> dashboard.
```

```
Sections in this file:
```

1. Settings (column name variations we recognise)
2. Loading the file
3. Detecting columns
4. Data quality checks
5. KPIs
6. Charts
7. The page (main)

```
"""
```

```
import io
```

```
import os
```

```
import re
```

```
import warnings
```

```
import pandas as pd
```

```
import streamlit as st
```

```
# -----
```

```
# 1. SETTINGS
```

```
# -----
```

```
# Names are written in lowercase without spaces or symbols.
```

```
# "Order Date", "order_date" and "ORDERDATE" all become "orderdate".
```

```
FIELD_ALIASES = {
```

```
    "revenue": ["revenue", "sales", "totalsales", "salesamount", "totalrevenue",  
                "amount", "turnover", "income"],
```

```
    "expenses": ["expenses", "expense", "cost", "costs", "totalcost",  
                "totalexpenses", "spending"],
```

```
    "profit": ["profit", "netprofit", "grossprofit"],
```

```
    "date": ["date", "orderdate", "transactiondate", "invoicedate", "saledate", "
```

```
    "product": ["product", "productname", "item", "itemname", "sku"],
```

```
    "customer": ["customer", "customername", "client", "clientname", "buyer"],
```

```
    "category": ["category", "productcategory", "type", "segment"],
```

```
    "department": ["department", "dept", "division", "team"],
```

```
    "quantity": ["quantity", "qty", "unitssold", "units", "orderquantity"],
```

```
    "supplier": ["supplier", "suppliername", "vendor", "vendorname"],
```

```
    "employee": ["employee", "employeenname", "staff", "salesperson", "salesrep"],
```

```
}
```

```

MAX_FILE_MB = 20
SAMPLE_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                             "sample_business_data.csv")

NUMBER_FIELDS = ["revenue", "expenses", "profit", "quantity"]

# Nice names for the screen
FIELD_LABELS = {
    "revenue": "Revenue / Sales", "expenses": "Expenses / Cost", "profit": "Profi
    "date": "Date", "product": "Product", "customer": "Customer",
    "category": "Category", "department": "Department", "quantity": "Quantity",
    "supplier": "Supplier", "employee": "Employee",
}

# -----
# 2. LOADING THE FILE
# -----

def load_file(name, data):
    """Read a CSV or Excel file from bytes. Raises ValueError with a friendly mes
    name = name.lower()

    if len(data) > MAX_FILE_MB * 1024 * 1024:
        raise ValueError(f"The file is larger than {MAX_FILE_MB} MB. Please uploa
            "file (for example, one year of data instead of ten).")

    try:
        if name.endswith(".csv"):
            df = None
            for encoding in ("utf-8", "utf-8-sig", "latin-1"):
                try:
                    df = pd.read_csv(io.BytesIO(data), encoding=encoding)
                    break
                except UnicodeDecodeError:
                    continue
            if df is None:
                raise ValueError("The file's text encoding could not be read. "
                    "Try re-saving it as CSV (UTF-8) from Excel.")
        elif name.endswith(".xlsx"):
            df = pd.read_excel(io.BytesIO(data), engine="openpyxl") # first shee
        elif name.endswith(".xls"):
            raise ValueError("Old Excel files (.xls) are not supported. Open the
                "Excel, choose Save As, and save it as .xlsx or .csv
        else:
            raise ValueError("Unsupported file type. Please upload a .csv or .xls
    except pd.errors.EmptyDataError:
        raise ValueError("The file is empty.")

```

```

except ValueError:
    raise
except Exception:
    raise ValueError("The file could not be read. Please check that it is a "
                      "valid Excel/CSV file and that it is not password-protected")

# Remove completely empty rows/columns and tidy the column names
df = df.dropna(how="all").dropna(axis=1, how="all")
df.columns = [str(c).strip() for c in df.columns]

if df.empty:
    raise ValueError("The file has no data rows. Make sure the first row contains
                      "column names and the data starts below it.")
return df

# -----
# 3. DETECTING COLUMNS
# -----
def simplify(name):
    """'Order Date' -> 'orderdate'"""
    return re.sub(r"^[a-z0-9]", "", str(name).lower())

def to_number(series):
    """Turn text like 'Rs. 1,200' into 1200. Bad values become NaN."""
    if pd.api.types.is_numeric_dtype(series):
        return series
    text = series.astype(str).str.replace(r"(?i)rs\.|pkr|usd|[$€£,\s]", "", regex=True)
    return pd.to_numeric(text, errors="coerce")

def to_date(series):
    """Turn text into dates. Bad values become NaT."""
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        return pd.to_datetime(series, errors="coerce")

def mostly_valid(converted, original):
    """True if at least half of the filled cells converted successfully."""
    filled = original.notna().sum()
    return filled > 0 and converted.notna().sum() / filled >= 0.5

def detect_columns(df):
    """Return {'revenue': 'Sales', 'date': 'Order Date', ...} for what we can find"""

```

```

found = {}
used = set()
simple_names = {col: simplify(col) for col in df.columns}

def looks_right(field, col):
    if field in NUMBER_FIELDS:
        return mostly_valid(to_number(df[col]), df[col])
    if field == "date":
        return mostly_valid(to_date(df[col]), df[col])
    return True

# Pass 1: exact matches. Pass 2: name contains an alias.
for exact_only in (True, False):
    for field, aliases in FIELD_ALIASES.items():
        if field in found:
            continue
        for col, simple in simple_names.items():
            if col in used:
                continue
            if exact_only:
                matched = simple in aliases
            else:
                matched = any(a in simple for a in aliases if len(a) >= 4)
            if matched and looks_right(field, col):
                found[field] = col
                used.add(col)
                break
    return found

def prepare_data(df, cols):
    """Copy of the data with numbers and dates converted to proper types."""
    clean = df.copy()
    for field in NUMBER_FIELDS:
        if field in cols:
            clean[cols[field]] = to_number(df[cols[field]])
    if "date" in cols:
        clean[cols["date"]] = to_date(df[cols["date"]])
    return clean

@st.cache_data(show_spinner="Analysing your file...")
def analyse_file(name, data):
    """Load and prepare the file once, so the page stays fast."""
    df = load_file(name, data)
    cols = detect_columns(df)
    clean = prepare_data(df, cols)

```

```

    return df, cols, clean

# -----
# 4. DATA QUALITY
# -----
def check_data_quality(df, cols):
    """Return a list of plain-language problems found in the file."""
    problems = []

    missing = int(df.isna().sum().sum())
    if missing:
        worst = df.isna().sum().sort_values(ascending=False)
        worst = [f"{c} ({n})" for c, n in worst.items() if n > 0][:3]
        problems.append(f"{missing} empty cells found. Most affected columns: "
                        f"{'', ' '.join(worst)}")

    duplicates = int(df.duplicated().sum())
    if duplicates:
        problems.append(f"{duplicates} duplicate rows found (exact copies of anot
                        "They may be counted twice in your totals.")

    if "date" in cols:
        col = cols["date"]
        bad = int((df[col].notna() & to_date(df[col]).isna()).sum())
        if bad:
            problems.append(f"{bad} values in '{col}' are not valid dates and wer
                            "in the trend charts.")

    for field in NUMBER_FIELDS:
        if field in cols:
            col = cols[field]
            numbers = to_number(df[col])
            bad = int((df[col].notna() & numbers.isna()).sum())
            if bad:
                problems.append(f"{bad} values in '{col}' are text, not numbers,
                                "left out of the calculations.")
            if field in ("revenue", "expenses", "quantity"):
                negative = int((numbers < 0).sum())
                if negative:
                    problems.append(f"{negative} negative values in '{col}'. "
                                    "Please check if these are returns or mistake

    return problems

# -----
# 5. KPIs

```

```
# -----
def calculate_kpis(clean, cols):
    """Return a list of (label, value, note). Only KPIs we have data for."""
    kpis = []
    revenue = clean[cols["revenue"]].sum() if "revenue" in cols else None
    expenses = clean[cols["expenses"]].sum() if "expenses" in cols else None

    profit, profit_note = None, ""
    if "profit" in cols:
        profit = clean[cols["profit"]].sum()
    elif revenue is not None and expenses is not None:
        profit = revenue - expenses
        profit_note = "Revenue minus Expenses"

    if revenue is not None:
        kpis.append(("Total Revenue", f"{revenue:,.0f}", ""))
    if expenses is not None:
        kpis.append(("Total Expenses", f"{expenses:,.0f}", ""))
    if profit is not None:
        kpis.append(("Total Profit", f"{profit:,.0f}", profit_note))
    if profit is not None and revenue:
        kpis.append(("Profit Margin", f"{profit / revenue * 100:.1f}%", "Profit ÷"))
    if "quantity" in cols:
        kpis.append(("Quantity Sold", f"{clean[cols['quantity']].sum():,.0f}", ""))
    kpis.append(("Records", f"{len(clean):,}", "Rows in the file"))
    return kpis
```

```
# -----
# 6. CHARTS
# -----
def show_trend_chart(clean, cols):
    """Monthly Revenue / Expenses / Profit line chart (needs a date column)."""
    if "date" not in cols:
        return
    series_cols = {}
    for field in ("revenue", "expenses", "profit"):
        if field in cols:
            series_cols[FIELD_LABELS[field]] = cols[field]
    if not series_cols:
        return

    data = clean.dropna(subset=[cols["date"]])
    month = data[cols["date"]].dt.to_period("M").dt.to_timestamp()
    monthly = data.groupby(month)[list(series_cols.values())].sum()
    monthly.columns = list(series_cols.keys())
```

```

# If there is no profit column, calculate it from revenue and expenses
if "profit" not in cols and "revenue" in cols and "expenses" in cols:
    monthly["Profit"] = monthly["Revenue / Sales"] - monthly["Expenses / Cost"]

if len(monthly) < 2:
    st.info("The trend chart needs data from at least 2 different months.")
    return
st.subheader("Monthly Trend")
st.line_chart(monthly)

def show_top_chart(clean, cols, field):
    """Top 10 bar chart of revenue by product / category / department."""
    if field not in cols or "revenue" not in cols:
        return
    totals = (clean.groupby(cols[field])[cols["revenue"]].sum()
               .sort_values(ascending=False).head(10))
    if len(totals) < 2:
        return
    st.subheader(f"Revenue by {FIELD_LABELS[field]} (top 10)")
    st.bar_chart(totals)

# -----
# 7. THE PAGE
# -----
def main():
    st.set_page_config(page_title="Business Analytics Tool", page_icon="🇮🇹", layout="center")
    st.title("🇮🇹 Business Analytics Tool")
    st.caption("Upload your business data and get an instant dashboard.")

    with st.expander("How to use", expanded=True):
        st.markdown(
            "1. Upload a **.csv** or **.xlsx** file.\n"
            "2. The first row must contain column names (e.g. Date, Product, Sale\n"
            "3. Column names do not need to match exactly. The tool finds them au\n"
            )

    st.caption("🔒 Your file is processed only during your session and is not saved")

    uploaded = st.file_uploader("Upload your file", type=["csv", "xlsx", "xls"])
    if st.button("Try with sample data"):
        st.session_state["use_sample"] = True
    if uploaded is not None:
        st.session_state["use_sample"] = False

    if uploaded is not None:

```

```

        file_name, file_bytes = uploaded.name, uploaded.getvalue()
    elif st.session_state.get("use_sample"):
        if not os.path.exists(SAMPLE_FILE):
            st.error("The sample file was not found. Please upload your own file.")
            st.stop()
        with open(SAMPLE_FILE, "rb") as f:
            file_name, file_bytes = "sample_business_data.csv", f.read()
        st.info("Showing sample data. This is demo data made for testing, "
                "not real business results.")
    else:
        st.stop()

    try:
        df, cols, clean = analyse_file(file_name, file_bytes)
    except ValueError as error:
        st.error(str(error))
        st.stop()

    # --- Detected fields ---
    st.subheader("Detected Columns")
    if cols:
        st.table(pd.DataFrame({
            "Field": [FIELD_LABELS[f] for f in cols],
            "Your column": list(cols.values()),
        }))
    else:
        st.warning("No known business columns were recognised (like Sales, Cost,
                    Please check your column names.)")

    # --- KPIs ---
    st.subheader("Key Numbers")
    kpis = calculate_kpis(clean, cols)
    for row_start in range(0, len(kpis), 4):
        row = kpis[row_start:row_start + 4]
        for box, (label, value, note) in zip(st.columns(4), row):
            box.metric(label, value, help=note or None)

    # --- Charts ---
    show_trend_chart(clean, cols)
    for field in ("product", "category", "department"):
        show_top_chart(clean, cols, field)

    # --- Data quality ---
    st.subheader("Data Quality")
    problems = check_data_quality(df, cols)
    if problems:
        for problem in problems:

```



```
        st.warning(problem)
    else:
        st.success("No problems found in this file.")

    # --- Raw data ---
    with st.expander("View uploaded data"):
        st.dataframe(df, width="stretch")

if __name__ == "__main__":
    main()
```